Assignment No	**11**
Name	**Satish Rathod**
Title	**Doubly Linked List**
Objective	**Insertion and Deletion**
Roll No	**MCA2649**

Source Code:-

```
//Name:- Satish Rathod
//Roll No:- 2649
//Doubly Linked List

class DNode
{

    // two pointers - refering to next and prev
    int data;
    DNode prev;
    DNode next;
    DNode(int data)
    {
        this.data=data;
        prev=null;
        next=null;
```

```
        }
}
class DoublyLinkedList
{
        // two nodes
        DNode head;
        DNode tail;
        // insert at the begining
        void insertFirst(int data)
        {
                DNode newNode= new DNode(data);
                // check whether the list is empty
                if(head==null)
                {
                        head=tail=newNode;
                        return;
                }
                newNode.next=head;
                head.prev=newNode;
                head = newNode;
        }

        // insert at the Last
        void insertLast(int data)
        {
                DNode newNode= new DNode(data);
```

```
        // check whether the list is empty
        if(head==null)
        {
                head=tail=newNode;
                return;
        }
        tail.next = newNode;
        newNode.prev = tail;;
        tail = newNode;
}


// display forward
void displayForward()
{
        DNode temp = head;
        System.out.println("Forward....");
        while(temp !=null)
        {
                System.out.print(temp.data+"<->");
                temp = temp.next;
        }
        System.out.println("Null");
}

// display Backward
```

```
void displayBackward()
{
        DNode temp = tail;
        System.out.println("Backward....");
        while(temp != null)
        {
                System.out.print(temp.data+"<->");
                temp = temp.prev;
        }
        System.out.println("Null");
}

// Count of Nodes
int count()
{
        DNode temp = head;
        int count=0;
        while(temp !=null)
        {
                count++;
                temp = temp.next;
        }
        return count;
}

// insert at a specific position
```

```
void insertAt(int data, int pos)
{
        int cnt = count();
        if(pos < -1 || pos+1 > cnt)
        {
                System.out.println("Invalid Position !!");
        }
        // insert in begining if pos is 1
        if( pos == 1)
        {
                insertFirst(data);
                return;
        }
        // insert at Last
        if( pos == cnt)
        {
                insertLast(data);
                return;
        }
        // traverse till a node before position
        DNode temp = head;
        for(int i=1;i<pos-1;i++)
        {
                temp=temp.next;
        }
        DNode newNode = new DNode(data);
```

```
        // assign prev and next of new node
        newNode.prev=temp;
        newNode.next=temp.next;
        temp.next.prev = newNode;
        temp.next = newNode;

}

void search(int key)
{
        int pos=1;
        DNode temp = head;
        while(temp!=null)
        {
                if(temp.data == key)
                {
                        System.out.println("Found at:"+pos);
                        return;
                }
                temp = temp.next;
                pos++;
        }
        System.out.println("Not Found");
}
```

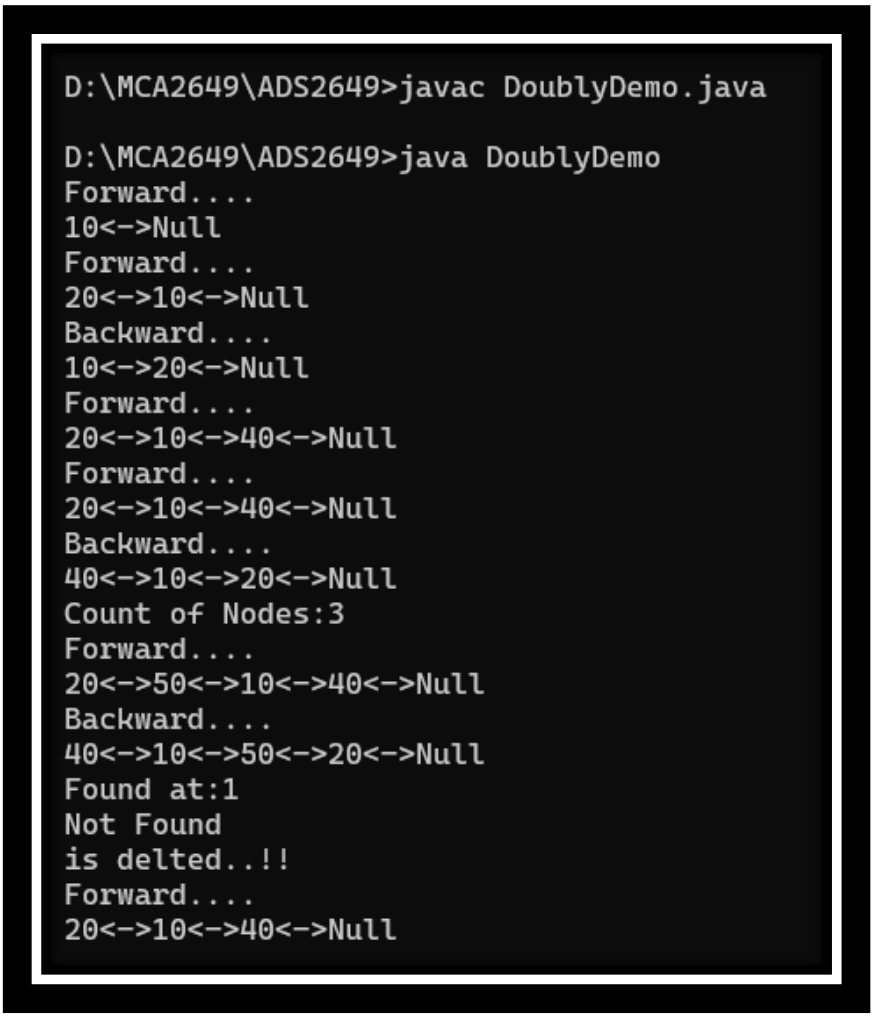
```
// delete by value
void delete(int key)
{
        DNode curr= head;
        while(curr != null && curr.data!=key)
        {
                curr = curr.next;

        }
        // not found
        if(curr == null)
        {
                System.out.println("Not Found To Delete!!!");
                return;
        }
        if(curr.prev != null)
        {
                curr.prev.next = curr.next;
        }
        else
        {
                head = curr.next;
        }
        // change previous
        if(curr.next != null)
        {
```

```
                // not tail
                curr.next.prev = curr.prev;
            }
            else
            {
                //tail
                tail = curr.prev;
            }
            System.out.println("is delted..!!");
        }
}
class DoublyDemo
{
    public static void main(String[] args)
    {
        DoublyLinkedList dl1 = new DoublyLinkedList();
        dl1.insertFirst(10);
        dl1.displayForward();
        dl1.insertFirst(20);
        dl1.displayForward();
        dl1.displayBackward();
        dl1.insertLast(40);
        dl1.displayForward();
        dl1.displayForward();
        dl1.displayBackward();
        System.out.println("Count of Nodes:"+ dl1.count());
```

```
            dl1.insertAt(50,2);
            dl1.displayForward();
            dl1.displayBackward();
            dl1.search(20);
            dl1.search(70);
            dl1.delete(50);
            dl1.displayForward();
        }
}
```

OUTPUT:-



```
D:\MCA2649\ADS2649>javac DoublyDemo.java

D:\MCA2649\ADS2649>java DoublyDemo
Forward....
10<->Null
Forward....
20<->10<->Null
Backward....
10<->20<->Null
Forward....
20<->10<->40<->Null
Forward....
20<->10<->40<->Null
Backward....
40<->10<->20<->Null
Count of Nodes:3
Forward....
20<->50<->10<->40<->Null
Backward....
40<->10<->50<->20<->Null
Found at:1
Not Found
is delted..!!
Forward....
20<->10<->40<->Null
```